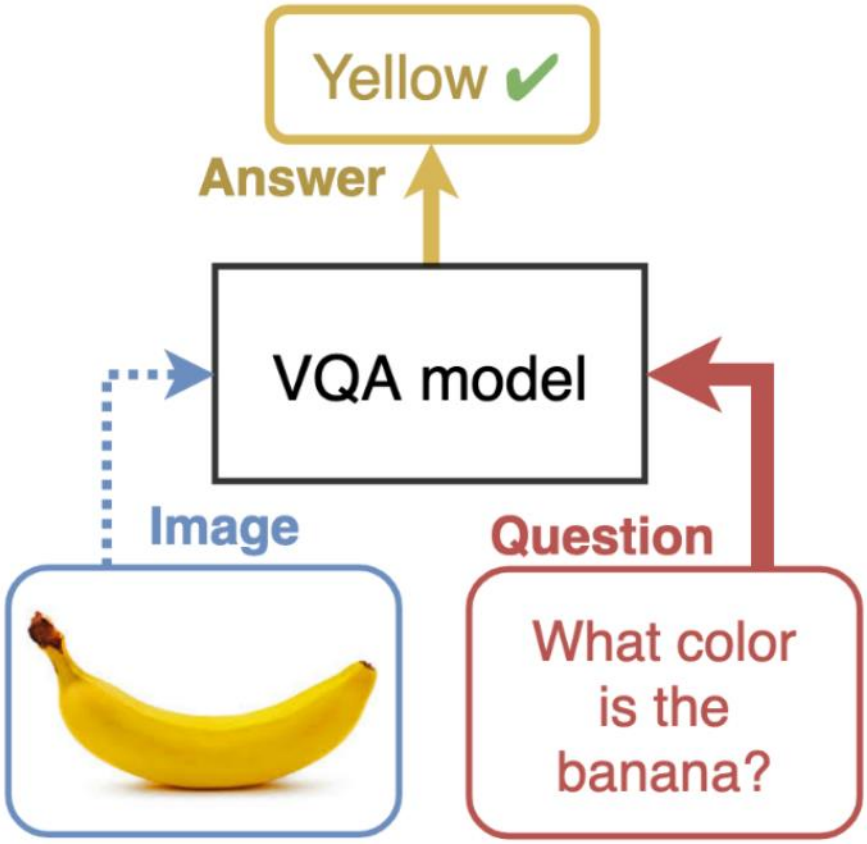
Alexandria University
Faculty of Engineering
Computer and Communication Program

CC484 Pattern Recognition

Assignment Four: Visual Questions Answering



Name	ID
Omar Walied Mohamed	7058
Hend Khaled Aly	6986
Habiba Mohamed Hefny	6939

Table of Contents

➢ Introduction

Visual Question Answering (VQA) is a task in the field of machine learning where machines are trained to answer open-ended questions about images. This task requires a fusion of computer vision and natural language processing techniques to enable machines to comprehend the visual content of an image and generate meaningful answers in response to diverse questions.

VQA systems aim to bridge the gap between visual perception and language understanding, enabling machines to reason about images and provide human-like responses. By combining the power of deep learning algorithms with the ability to process natural language, VQA systems have the potential to revolutionize various applications, such as image understanding, content retrieval, and human-machine interaction.

To tackle the language understanding aspect of VQA, natural language processing techniques come into play. Recurrent neural networks (RNNs) and transformer models have shown remarkable progress in understanding and generating human-like textual data. By leveraging these techniques, VQA systems can process and analyze the textual question in conjunction with the visual information from the image to generate accurate answers.

The potential applications of VQA are vast and diverse. It can be used in fields such as robotics, autonomous vehicles, virtual assistants, and augmented reality, where machines need to interact with and understand visual information in a human-like manner.

➢ Problem Description

In our project, we specifically focused on the VizWiz dataset, which was designed to train models that can assist visually impaired individuals. The VizWiz dataset was carefully curated to cater to the needs of visually impaired people. The creators of VizWiz introduced the concept of visual question answering (VQA) and developed the VizWiz-VQA dataset specifically for this population. In this dataset, blind individuals captured images and recorded spoken questions about them. Each visual question was accompanied by ten crowdsourced answers to provide a

diverse range of responses. This dataset helps them in understanding visual information. It presents real-world scenarios that are specifically relevant to their needs. As a result, VizWiz-VQA offers a valuable resource for training and evaluating VQA models that aim to enhance the visual perception and comprehension abilities of visually impaired individuals. In this assignment, we will explore the VizWiz dataset, analyze its characteristics, and delve into the methodologies used to train VQA models using this dataset. We aim to explore and analyze the effectiveness of various VQA approaches in addressing the challenges faced by visually impaired individuals. By training models on this dataset, we strive to improve the accessibility and inclusivity of visual information for this population. By understanding the challenges and intricacies of VQA for visually impaired individuals, we can contribute to the development of innovative solutions that improve their access to visual information and enhance their overall quality of life.

➢ Code Explaination and Screenshots:

First. we created a new kaggle notebook, added the dataset to it, installed the necessary dependencies and libraries (the clip pretrained models) for working with the (Contrastive Language-Image Pretraining) model.

```
!conda install --yes -c pytorch pytorch=1.7.1 torchvision cudatoolkit=11.0
!pip install ftfy regex tqdm
!pip install git+https://github.com/openai/CLIP.git
```

- The first line is for installing PyTorch and torchvision which are popular deep learning libraries for working with neural networks. It also installs the appropriate version of the CUDA toolkit, which is required for GPU acceleration.
- The second command installs additional python packages required by CLIP model like **ftfy** which is for handling and fixing text encoding issues, **regex** which is a library for advanced string pattern matching and manipulation. **tdqm** which is for creating progress bars and tracking the progress of code execution.
- The last command installs the CLIP model, developed by OpenAI. CLIP is a versatile model that learns visual and textual representations jointly. It can be

used for a variety of tasks, including image classification, object detection, and zero-shot learning.

By installing these dependencies and the CLIP model, the code prepares the environment for working with CLIP and enables the use of its powerful visual and textual understanding capabilities in machine learning applications.

Second. We imported some libraries and modules that are required for different functionalities and operations.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import numpy as np
import cv2
import matplotlib.pyplot as plt
from sklearn import metrics
from torch.autograd import Variable
import os
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim
import random
import clip
from torchvision.datasets import CIFAR100
from PIL import Image, ImageEnhance
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import pandas as pd
import json
from random import randint
from collections import Counter
```

- **Torch:** main PyTorch library which provides the foundational building blocks for creating and training neural networks.
- **Torch.nn:** contains various neural network layer classes and functions.
- **Torch.nn.functional:** functional module from PyTorch's neural network library. It provides functions that are commonly used in neural network operations, such as activation functions and loss functions.
- **Torchvision:** provides utility functions, datasets, and pre-trained models for computer vision tasks.
- **Numpy:** a fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and mathematical functions to operate on these arrays efficiently.
- **Cv2:** the OpenCV library, which is a popular computer vision library. It provides various functions and tools for image and video processing.

- **matplotlib.pyplot:** the pyplot module from the Matplotlib library, which is a plotting library in Python. It allows for the creation of visualizations, such as line plots, scatter plots, and histograms.
- **from sklearn import metrics:** the metrics module from the scikit-learn library, which provides various evaluation metrics for machine learning models, such as accuracy, precision, recall, and F1-score.
- **from torch.autograd import Variable:** This imports the Variable class from the autograd module in PyTorch. Variables were used in previous versions of PyTorch for automatic differentiation, but they have been deprecated in favor of tensors.
- **import os:** This imports the os module, which provides a way to interact with the operating system, such as accessing files and directories.
- **from sklearn.model_selection import train_test_split:** This imports the train_test_split function from the model_selection module in scikit-learn. It is used for splitting datasets into training and testing subsets.
- **from torch.utils.data import Dataset, DataLoader:** This imports the Dataset and DataLoader classes from PyTorch's data module. They are used for creating custom datasets and data loaders for efficient data loading during training.
- **torch.optim:** the optim module from PyTorch, which provides various optimization algorithms for training neural networks, such as stochastic gradient descent (SGD), Adam, and RMSprop.
- **random:** random module, which provides functions for generating random numbers, shuffling sequences, and making random choices.
- **clip:** Contrastive Language-Image Pretraining library. CLIP is a model developed by OpenAI that learns visual and textual representations jointly.
- **from torchvision.datasets import CIFAR100:** This imports the CIFAR100 dataset from the torchvision.datasets module. CIFAR100 is a commonly used benchmark dataset for image classification tasks.
- **from PIL import Image, ImageEnhance:** This imports the Image and ImageEnhance classes from the Python Imaging Library (PIL), which provides tools for opening, manipulating, and enhancing images.
- **tensorflow:** TensorFlow library, which is a popular deep learning framework for building and training neural networks.

- **from tensorflow.keras import layers:** This imports the layers module from TensorFlow's Keras API. It provides a high-level interface for building neural networks using TensorFlow.
- **pandas:** pandas library, which is used for data manipulation and analysis. It provides data structures and functions for handling structured data, such as tables and time series.
- **json:** json module, which provides functions for working with JSON (JavaScript Object Notation) data, such as reading and writing JSON files.
- **from random import randint:** This imports the randint function from the random module. It is used for generating random integers.
- **from collections import Counter:** This imports the Counter class from the collections module. Counter is a useful data structure for counting occurrences of elements in a list or iterable.

Third. We set the device variable to either "cuda" or "cpu" based on the availability of a CUDA-enabled GPU. If a CUDA-enabled GPU is available, then the device is set to "cuda", indicating that the GPU will be used for computations. If a CUDA-enabled GPU is not available, the device is set to "cpu", indicating that computations will be performed on the CPU.

We used 1xP1000 GPU.

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model, preprocess = clip.load("ViT-L/14@336px", device)
```

```
100%|████████████████████████████████████████| 891M/891M [00:06<00:00, 144MiB/s]
```

Then the CLIP model is loaded and its corresponding pre-processing function. The model is loaded onto the specified device (Px1000 GPU) based on the value of the device variable.
The function also returns a preprocess object, which is a pre-processing function used to normalize and transform input images before passing them to the model. By setting the device and loading the CLIP model, this code prepares the necessary components for using CLIP for visual and textual tasks, such as image classification or image-text matching.

Fourth. We read the contents of JSON file named train.json located at the specified file path (/kaggle/input/vizwiz/vizwiz_data_ver1/data/Annotations/train.json). then we assign the contents of the file to the variable json_data as a string. Then converting the string into an object depending on the JSON structure.

```
with open('/kaggle/input/vizwiz/vizwiz_data_ver1/data/Annotations/train.json') as file:
    json_data = file.read()

    data = json.loads(json_data)
```

Fifth. We processes the data extracted from the JSON file and generates two sets of **Unique** labels (labels1 and labels2).

labels1: the unique answer types (size was 4)

labels2: the unique answers (size was about 5860)

```
print(len(labels1))
print(len(labels2))
```

```
4
5860
```

The loop iterates through each item in the data object, which represents the parsed JSON data. For each item:

- Extract the value of the "answer_type" key from the item and append it to labels1.
- Retrieve the list of "answers" from the item.
- Initialize two empty lists, yes and maybe, which will be used to categorize the answers based on their "answer_confidence" attribute.
- For each answer in answers, it checks the value of the "answer_confidence" attribute. If it is "yes", the answer is appended to the yes list; otherwise, it is appended to the maybe list.

- Next, it checks if the yes list contains any answers. If it does, the use list is assigned the values from the yes list; otherwise, it is assigned the values from the maybe list.
- Then creating a common list and iterates through the use list to extract the "answer" attribute of each answer and append it to the common list.
- Count the occurrences of each item in the common list and retrieves the most common items. The most common item(s) with the highest count are stored in the most_repeated_items list.
- The labels2 list appends the first item from the most_repeated_items list (that is sorted primarily on number of occurrences then secondarily in ascending order).
- After processing all the items, the code uses the Counter class to count the occurrences of each label in labels1 and labels2. The labels are then sorted in descending order of their counts and, in the case of ties, in ascending order of their values.

```
labels1=[]
labels2=[]
for item in data:
    labels1.append(item['answer_type'])
    answers = item['answers']
    yes=[]
    maybe=[]
    for answer in answers:
        if answer['answer_confidence']=="yes":
            yes.append(answer)
        else:
            maybe.append(answer)
    use=[]
    if len(yes)>0:
        use=yes
    else:
        use=maybe
    common=[]
    i=0
    for t in use:
        common.append(use[i]['answer'])
        i=i+1

    counter = Counter(common)
    most_common = counter.most_common()
    max_count = most_common[0][1]
    most_repeated_items = [item for item, count in most_common if count == max_count]
    labels2.append(most_repeated_items[0])
```

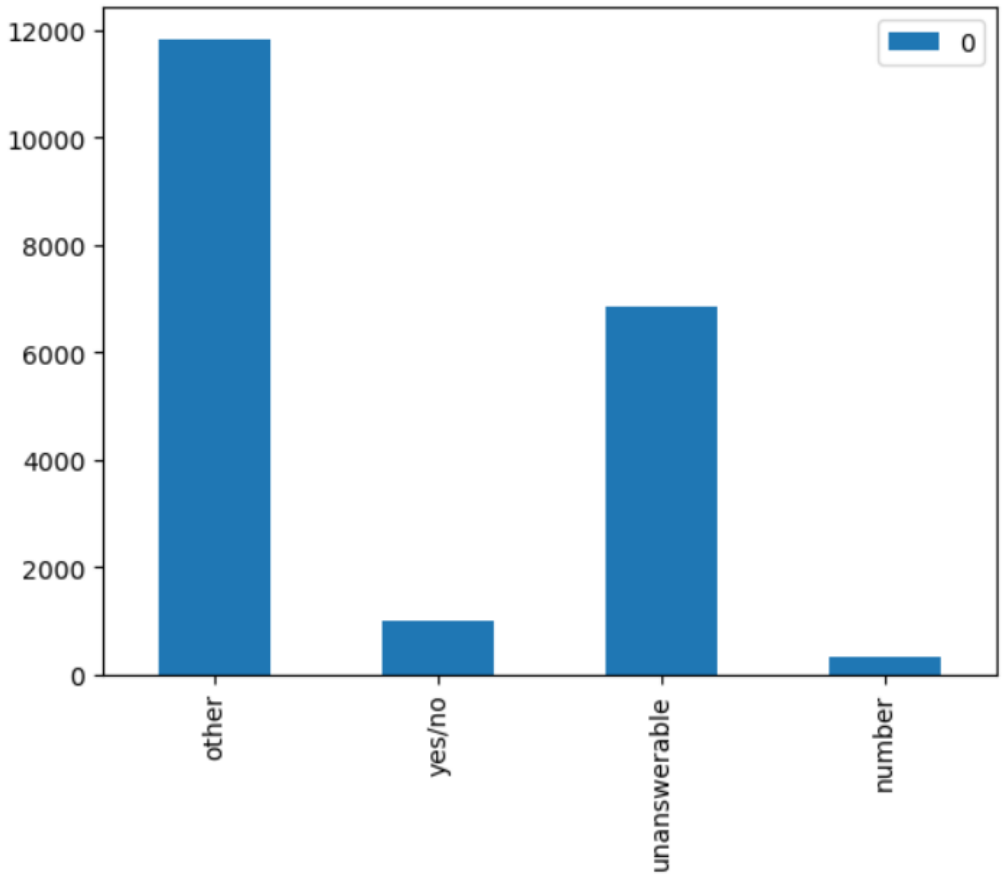
```
counter = Counter(labels1)
labels1 = sorted(list(dict.fromkeys(labels1)))
labels1 = sorted(labels1, key=lambda x: (-counter[x], x))
counter = Counter(labels2)
labels2 = sorted(list(dict.fromkeys(labels2)))
labels2 = sorted(labels2, key=lambda x: (-counter[x], x))
```

Sixth. We create a bar plot to visualize the frequency of each answer type in label string.

```
letter_counts = Counter(labelstr)
df = pd.DataFrame.from_dict(letter_counts, orient='index')
df.plot(kind='bar')
```

- 1st step is counting the occurrences of each letter in labelstr string, The result is stored in the letter_counts variable, which is a dictionary-like object with letters as keys and their corresponding counts as values.
- Then creating a pandas DataFrame from the letter_counts dictionary. The DataFrame has two columns: the letters (taken from the dictionary keys) and their counts (taken from the dictionary values). The orient='index' argument specifies that the dictionary keys should be used as the index of the DataFrame.

- Then a bar plot is generated where the x-axis represents the letters, and the y-axis represents the counts.



Seventh. We define a custom dataset class named myDataset that inherits from the torch.utils.data.Dataset class. This class allows you to retrieve individual data samples and their corresponding labels in a format that can be easily used for training or evaluation in a machine learning model.

This class has 3 methods, 1st the __init__ method, by which the class initializes the dataset by taking in an array of data (array), labels1, labels2, and a boolean flag tr (which stands for "train") indicating whether the dataset is used for training or

not. The data, labels, and the flag are stored as attributes of the class.

```
class myDataset(Dataset):
  def __init__(self, array,labels1,labels2,tr=False):
    self.data = array
    self.labels1=labels1
    self.labels2=labels2
    self.tr=tr
```

2nd The __getitem__ method is implemented to retrieve an item from the dataset given an index. In this case, it retrieves the item at the specified index from the data array. The item contains various attributes such as answerable, image, question, answer_type, and answers. It processes the answers to categorize them into the yes and maybe lists based on their "answer_confidence" attribute.

- It performs data augmentation on the image data using random transformations such as flipping, rotating, and adjusting brightness, if the tr flag is True.
- It pre-processes the image data and question text by applying the CLIP model's pre-processing functions (preprocess) and converts them into the appropriate tensor format for further processing.
- It uses the CLIP model (model) to encode the image and text inputs into feature representations.
- It concatenates the encoded image and text features along the channel dimension to create a single data tensor (datrain) representing the input data for the model.
- It computes the most common answers from the common list and finds the index of the most repeated answer based on the same criterion as before that is present in labels2.

- It encodes the answer type and most repeated answer labels into two one-hot encoded tensors (datalabel1 and datalabel2).
- Finally, it returns the datrain, datalabel1, and datalabel2 tensors as the output of the __getitem__ method.

```
def __getitem__(self, index):
        item=self.data[index]
        answerable = item['answerable']
        image = item['image']
        question = item['question']
        answer_type = item['answer_type']
        answers = item['answers']
        answer1  = answers[0]['answer']
        answer2  = answers[1]['answer']
        answer3  = answers[2]['answer']
        answer4  = answers[3]['answer']
        answer5  = answers[4]['answer']
        answer6  = answers[5]['answer']
        answer7  = answers[6]['answer']
        answer8  = answers[7]['answer']
        answer9  = answers[8]['answer']
        answer10 = answers[9]['answer']

        yes=[]
        maybe=[]
        for answer in answers:
            if answer['answer_confidence']=="yes":
                yes.append(answer)
            else:
                maybe.append(answer)
        use=[]
```

```
if len(yes)>0:
    use=yes
else:
    use=maybe
common=[]
i=0
for t in use:
    common.append(use[i]['answer'])
    i=i+1

with torch.no_grad():
    image_data = Image.open('/kaggle/input/vizwiz/vizwiz_data_ver1/data/Images/'+image)
    if(self.tr):
        zz=random.randint(0,6)
        if(zz==0):
            image_data = image_data.transpose(Image.FLIP_LEFT_RIGHT)
        elif(zz==1):
            image_data = image_data.transpose(Image.FLIP_TOP_BOTTOM)
        elif(zz==2):
            image_data = image_data.transpose(Image.ROTATE_90)
        elif(zz==3):
            enhancer = ImageEnhance.Brightness(image_data)
            image_data = enhancer.enhance(1.5)
        elif(zz==4):
            enhancer = ImageEnhance.Brightness(image_data)
            image_data = enhancer.enhance(0.5)
    image_input = preprocess(image_data).unsqueeze(0).to(device)
    text_input = clip.tokenize([question]).to(device)
```

```
with torch.no_grad():
    image_features = model.encode_image(image_input)
    text_in_features = model.encode_text(text_input)
    channel1=torch.tensor (image_features).float().clone().detach()
    channel2=torch.tensor (text_in_features).float().clone().detach()
datrain = torch.cat((channel1.unsqueeze(1), channel2.unsqueeze(1)), dim=1).clone().detach().to(device)

counter = Counter(common)
most_common = counter.most_common()

max_count = most_common[0][1]
most_repeated_items = [item for item, count in most_common if count == max_count]
minimum=np.inf
for x in most_repeated_items:
    if x in labels2:
        index = labels2.index(x)
        if index<minimum:
            minimum=index
anstype=labels1.index(answer_type)

l1 = [0] * 4
l1[int(anstype)] = 1

l2 = [0] * len(labels2)
l2[int(minimum)] = 1


datalabel1=torch.tensor([l1]).clone().detach().to(device)
datalabel2=torch.tensor([l2]).clone().detach().to(device)

return datrain, datalabel1, datalabel2
```

3rd The __len__ method returns the length of the dataset, which corresponds to the total number of examples or data points in the dataset.

```
def __len__(self):
    return len(self.data) # of how many examples(images?) you have
```

Eighth. We build our neural network model that is inherited from the torch.nn.Module class. The model architecture consists of several layers takes an input tensor, applies linear transformations, activation functions, normalization, and dropout, and produces two outputs (label1 and label2) based on the defined architecture.

- The __init__ function initializes the model and defines its architecture. It takes an argument outl, which represents the size of the output layer.
- The model consists of several linear layers (self.linear1, self.linear2, self.linear3, self.linear4) with different input and output sizes. These layers are used to perform linear transformations on the input data.

- The model also includes dropout regularization (self.dropout) with a dropout rate of 0.5, which randomly sets elements of the input tensor to zero during training to prevent overfitting.
- Activation functions (self.activation, self.activation2) are applied after certain layers to introduce non-linearity into the network. The ReLU activation function (nn.ReLU()) is used for most layers, and the sigmoid activation function (nn.Sigmoid()) is used for the self.activation2 layer.
- The model includes a softmax function (self.softmax) for the final output layer. Softmax converts the output values into probabilities that sum up to 1.
- The model also includes a layer normalization (self.norm) operation, which normalizes the activations of the linear layer.
- There are two output layers (self.output_layer1 and self.output_layer2) that produce the final outputs of the model. self.output_layer1 has an output size of 4, while self.output_layer2 has an output size of outl, which is provided as an argument during initialization.
- The forward function defines the forward pass of the model. It takes an input tensor x, performs the necessary operations, and returns two outputs label1 and label2.
- The input tensor x is reshaped to have a size of (batch_size, sequence_length, embedding_size).
- x is passed through the linear layers (self.linear1) and the output is stored in x.
- x is then normalized (self.norm), passed through dropout (self.dropout), and activated using ReLU (self.activation).
- The output of self.output_layer1 is calculated and stored in label1.
- label1 is further processed through self.linear2, activated using sigmoid (self.activation2), and stored in y.
- The output of self.output_layer2 is calculated and stored in label2.
- label2 is multiplied element-wise with y to adjust the output.
- Finally, label1 and label2 are returned as the outputs of the forward pass.

```python
class Model(nn.Module):
    def __init__(self,outl):
        super(Model, self).__init__()
       # Linear layers
        self.linear1 = nn.Linear(768 * 2, 256)
        self.linear2 = nn.Linear(4, outl)
        self.linear3 = nn.Linear(256,512)
        self.linear4 = nn.Linear(512,256)

        self.dropout = nn.Dropout (p=0.5)
        self.activation = nn.ReLU()
        self.activation2 = nn.Sigmoid()
        self.softmax =nn.Softmax ()
        # Normalization layer
        self.norm = nn.LayerNorm(256)

        # Attention layer
        self.attention = nn.MultiheadAttention(embed_dim=256, num_heads=4)

        # Output layer
        self.output_layer1 = nn.Linear(256,4)
        self.output_layer2 = nn.Linear(256,outl)
def forward(self, x):
    x = torch.tensor(x).clone().detach().to(device)
    # Reshape input to (batch_size, sequence_length, embedding_size)
    x = x.view(x.size(0), -1, 768 * 2)
    # Apply linear layers
    x = self.linear1(x)

    # Apply normalization layer
    x = self.norm(x)
    x = self.dropout(x)
    x = self.activation(x)

    # Apply output layer
    label1 = self.output_layer1(x)
    y = self.linear2(label1)
    y = self.activation2(y)
    label2 = self.output_layer2(x)
    label2=label2*y
    return label1,label2
```

Ninth. We perform data splitting and creates dataloaders for training, validation, and testing. Where we prepare the data for training a machine learning model, and we also create dataloaders to effciently load the data during training, validation, and testing phases. (we set a constant random variable 42 and stratify on the answer types)

```
X_train_temp, X_test, y_train_temp, y_test = train_test_split(data, labelstr, test_size=0.05, stratify=labelstr, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train_temp, y_train_temp, test_size=0.3, stratify=y_train_temp, random_state=42)
customDataset=myDataset(data,labels1,labels2)
customDataset_train=myDataset(X_train,labels1,labels2,True)
customDataset_val=myDataset(X_val,labels1,labels2,False)
customDataset_test=myDataset(X_test,labels1,labels2,False)
whole_dataloader = DataLoader(customDataset, batch_size=256,shuffle=True, num_workers=0)
train_dataloader = DataLoader(customDataset_train, batch_size=256,shuffle=True, num_workers=0)
val_dataloader = DataLoader(customDataset_val, batch_size=256,shuffle=True, num_workers=0)
test_dataloader = DataLoader(customDataset_test, batch_size=256,shuffle=True, num_workers=0)
```

- train_test_split: is a function from the sklearn.model_selection module that splits the data (data and labelstr) into training and testing sets. The test_size parameter specifies the proportion of the data to be included in the testing set. The stratify parameter ensures that the class distribution is preserved in the splits, and the random_state parameter ensures reproducibility of the splits. The resulting splits are assigned to X_train_temp, X_test, y_train_temp, and y_test.
- Another round of splitting is performed using train_test_split to further split the training set (X_train_temp and y_train_temp) into training and validation sets. The test_size parameter specifies the proportion of the data to be included in the validation set. The stratify parameter ensures that the class distribution is preserved, and the random_state parameter ensures reproducibility. The resulting splits are assigned to X_train, X_val, y_train, and y_val.
- Four instances of the myDataset class are created: customDataset, customDataset_train, customDataset_val, and customDataset_test. These instances are initialized with different inputs. customDataset is initialized with the original data (data, labels1, labels2). customDataset_train, customDataset_val, and customDataset_test are initialized with the corresponding splits (X_train, X_val, X_test) and the label information (labels1, labels2). The tr parameter is set to True for customDataset_train to indicate data augmentation.
- Four DataLoader objects are created: whole_dataloader, train_dataloader, val_dataloader, and test_dataloader. These dataloaders are used to load the

data in batches during training, validation, and testing. The batch_size parameter specifies the number of samples per batch, shuffle is set to True to shuffle the data during each epoch, and num_workers controls the number of subprocesses to use for data loading.

Tenth. We define a function called run_model that runs the training or evaluation loop for the given model on a specified dataloader. The function takes the following inputs:

- model: The model to be trained or evaluated.
- dataloader: The dataloader containing the input data and labels.
- optimizer: The optimizer used for updating the model's parameters.
- train: A boolean value indicating whether the model should be trained (True) or evaluated (False).

We check the train value to determine whether to train or evaluate.

- If train is True, the model is set to training mode using model.train(). Otherwise, it is set to evaluation mode using model.eval().
- Variables **totalacc**, **pred**, and **labels** are initialized. totalacc keeps track of the total accuracy, while pred and labels store the predicted and actual labels, respectively.
- The nn.CrossEntropyLoss() is used to compute the lossdrom the soft maxed output and the hot encoded label. The total loss is initialized as 0.

The loop iterates over the batches in the dataloader. For each batch:

- The gradients of the optimizer are set to zero using optimizer.zero_grad().
- The model is forward-propagated on the input data (data) to obtain the outputs (output1 and output2).
- The loss is computed based on the outputs and corresponding labels (label1 and label2).
- The loss is accumulated into the total_loss.
- If the train is True, the gradients are computed, and the optimizer's step() function is called to update the model's parameters. Additionally,

scheduler.step(loss_) is called to adjust the learning rate based on the loss value it decys the learning rate on plateux (fixed loss).

- The predicted outputs (output1 and output2) and corresponding labels (label1 and label2) are appended to pred and labels, respectively.
- The accuracy is calculated by comparing the predicted label with the actual label for each sample in label2.
- The accuracy and loss for the current batch are printed.
- Finally, the function returns the labels, pred, average loss per batch, and total accuracy.

```
def run_model(model,dataloader, optimizer,train = True ):
    if train:
        model.train()
    else:
        model.eval()
    totalacc=0
    pred = []
    labels = []
    loss = nn.CrossEntropyLoss()
    total_loss = 0
    for (data, label1, label2) in dataloader:
        optimizer.zero_grad()
        output1,output2 = model(data)
        loss_ = loss(torch.squeeze(output1.float()), torch.squeeze(torch.tensor(label1).to(device).float()))
        loss_ += loss(torch.squeeze(output2.float()), torch.squeeze(torch.tensor(label2).to(device).float()))
```

Eleventh. Perform training and evaluation to the model using a specified number of epochs using training and validation data. It uses an optimizer and scheduler to update the model's parameters and adjust the learning rate. The training and validation loss and accuracy values are recorded for each epoch.

- The mymodel is initialized with the number of output classes (len(labels2)) and moved to the specified device.
- Lists train_accuracy, val_accuracy, train_loss, and val_loss are initialized to keep track of the training and validation accuracy and loss values.
- An optimizer is defined using torch.optim.Adam with an initial learning rate of 0.01 and weight decay of 0.1.
- A scheduler is defined using torch.optim.lr_scheduler.ReduceLROnPlateau to adjust the learning rate based on the train loss. It reduces the learning rate by a factor of 0.7 if the validation loss plateaus for 5 consecutive epochs.

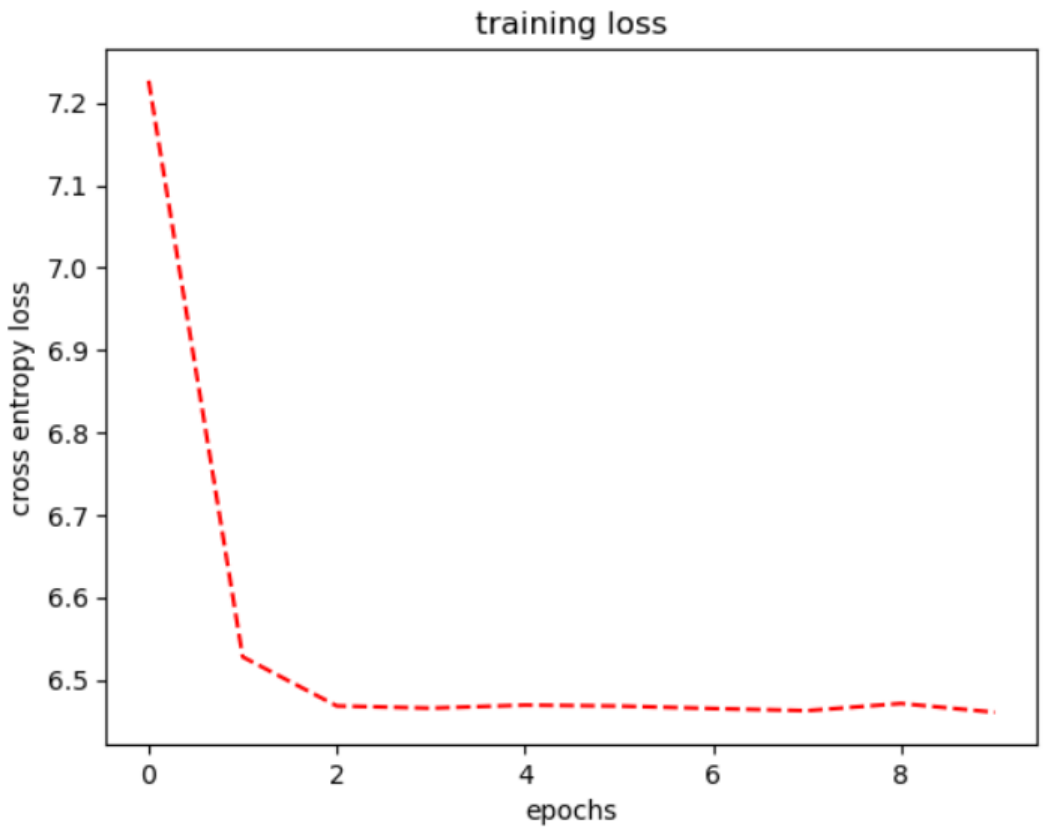
The training loop iterates for the specified number of epochs (epoch). For each epoch:

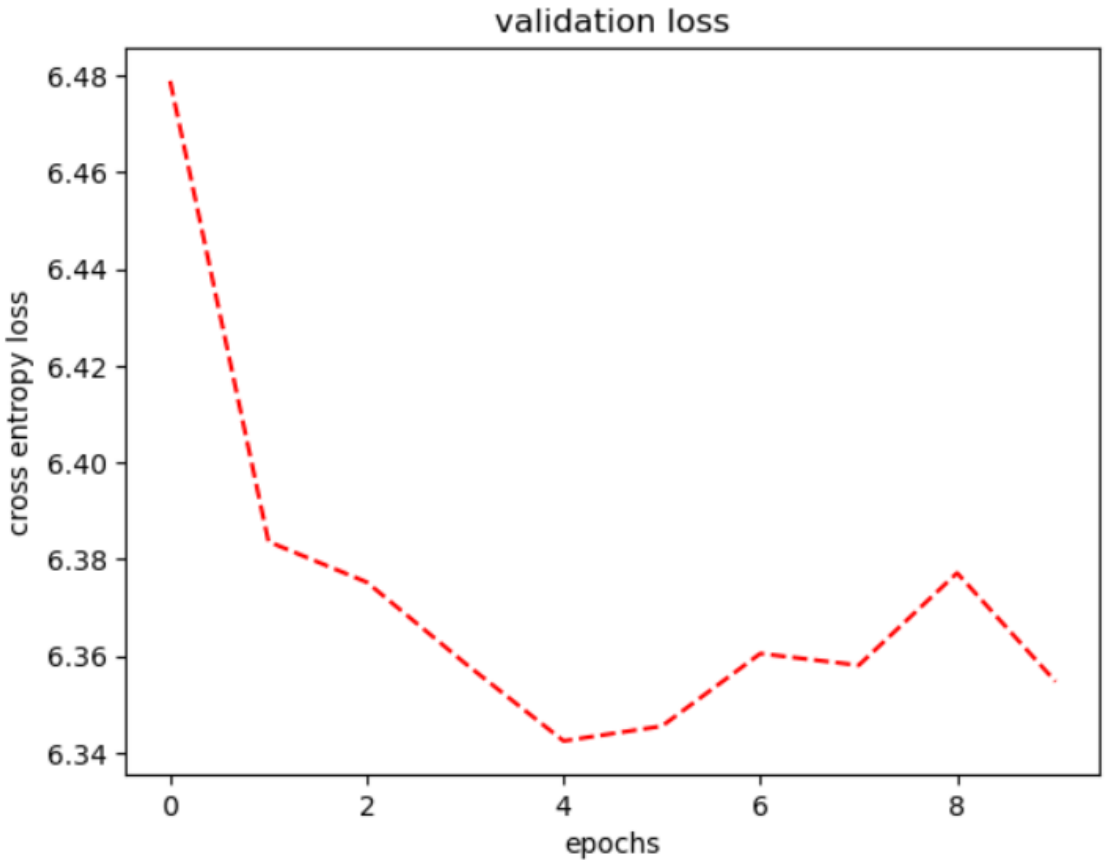
- The run_model function is called with the mymodel and the training dataloader (train_dataloader). This performs the training on the model, computes the loss, and returns the predicted labels, actual labels, loss, and accuracy.
- The same process is repeated with the validation dataloader (val_dataloader) by setting train=False.
- The training and validation loss and accuracy values are printed and appended to their respective lists.
- After the training loop is complete, the code prints the total loss, accuracy, validation loss, and validation accuracy for each epoch.

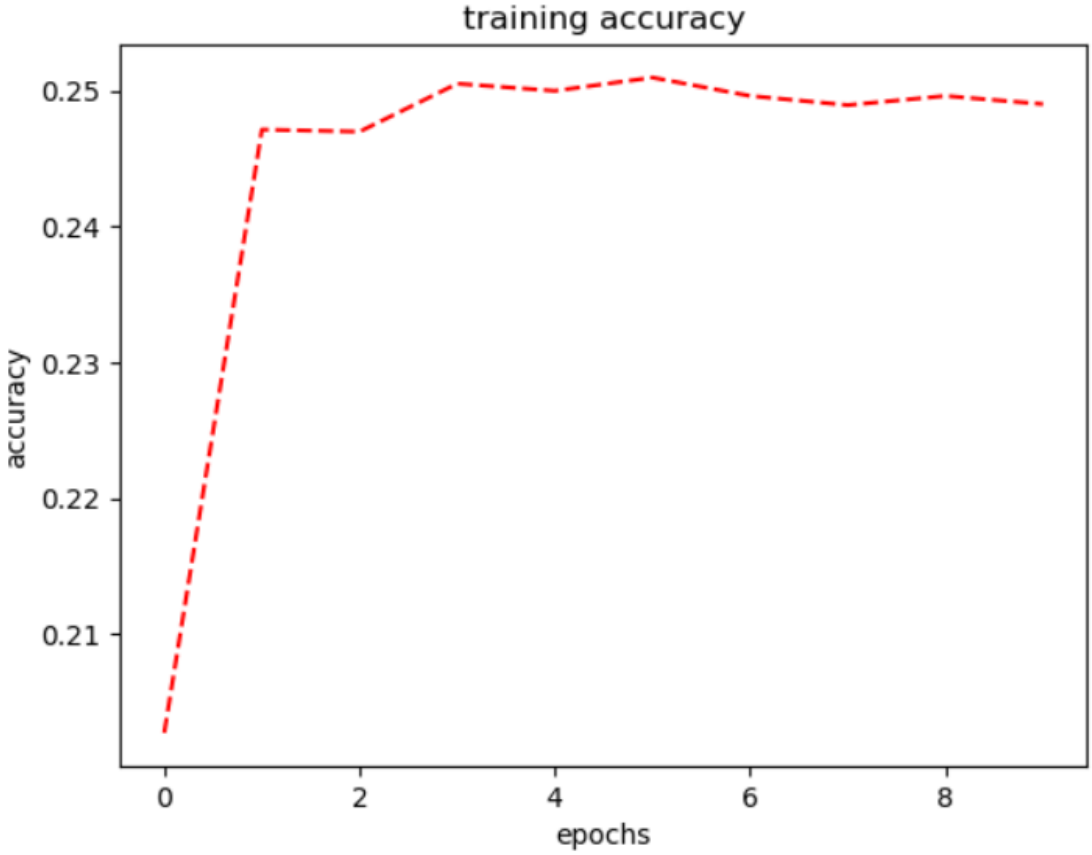
```
mymodel=Model(len(labels2)).to(device)
epoch = 10
train_accuracy=[]
val_accuracy=[]
train_loss=[]
val_loss=[]
optimizer = torch.optim.Adam(mymodel.parameters(), lr=0.01, weight_decay=.1)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5, factor=.7, threshold=1e-4)

for e in range(epoch):
    labels,pred,loss,totalacc=run_model(mymodel,train_dataloader,optimizer)
    print("train passed ----------------------------------------------------------------------------------------------------")
    labels,pred,loss2,totalacc2=run_model(mymodel,val_dataloader,optimizer,False)

  # calculate acc, f1 score, recall ......
    print("epoch passed ----------------------------------------------------------------------------------------------------")
    print("total loss",loss)
    train_loss.append(loss)
    print("train accuracy",totalacc/len(y_train))
    train_accuracy.append(totalacc/len(y_train))
    print("total val loss",loss2)
    val_loss.append(loss2)
    print("accuracy val",totalacc2/len(y_val))
    val_accuracy.append(totalacc2/len(y_val))
```

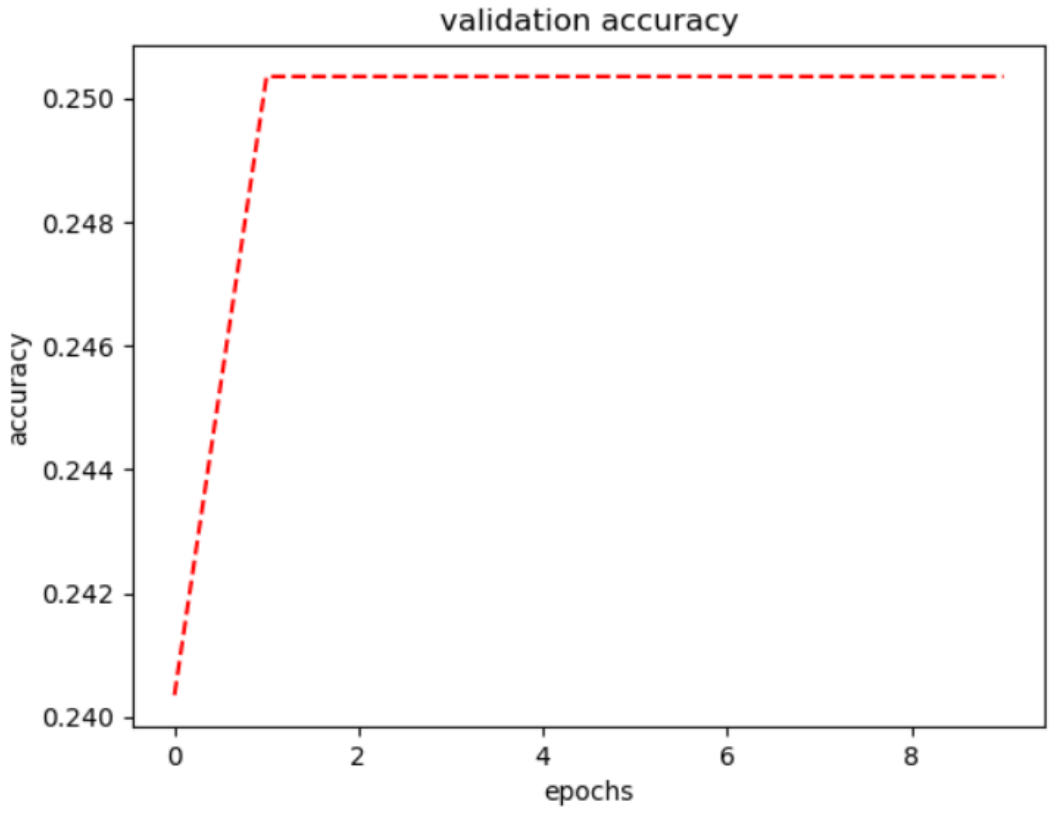
Twelfth. We visually represent the training loss (epochs vs cross entropy loss), validation loss (epochs vs cross entropy loss), training accuracy (epochs vs accuracy) and validation accuracy (epochs vs accuracy).

```
import matplotlib.pyplot as plt

def plot_broken_line(data,y,t):
    # Generate x-axis values as indices of the list
    x = range(len(data))
    # Plot the data as a broken line with red dashed line style
    plt.plot(x, data, 'r--')
    plt.xlabel('epochs')
    plt.ylabel(y)
    plt.title(t)
    plt.show()

plot_broken_line(train_loss,"cross entropy loss","training loss")
plot_broken_line(val_loss,"cross entropy loss","validation loss")
plot_broken_line(train_accuracy,"accuracy","training accuracy")
plot_broken_line(val_accuracy,"accuracy","validation accuracy")
```


training loss
cross entropy loss
7.2
7.1
7.0
6.9
6.8
6.7
6.6
6.5
0
2
4
6
8
epochs


validation loss
cross entropy loss
6.48
6.46
6.44
6.42
6.40
6.38
6.36
6.34
0
2
4
6
8
epochs


training accuracy
0.25
0.24
0.23
0.22
0.21
accuracy
0
2
4
6
8
epochs


validation accuracy
0.250
0.248
0.246
0.244
0.242
0.240
accuracy
0
2
4
6
8
epochs

Thirteenth. We evaluate the performance of the trained model on the test dataset by calculating the loss and accuracy metrices.

- call the run_model function with the trained model (mymodel) and the test dataloader (test_dataloader). The train parameter is set to False to indicate that it is not the training phase.
- The function returns the predicted labels (pred), true labels (labels), the calculated loss (loss3), and the total accuracy (totalacc3) on the test dataset.
- After running the model on the test dataset, the code prints the message "epoch passed".
- It then prints the total loss (loss3) obtained on the test dataset.
- Finally, it calculates and prints the accuracy by dividing the total accuracy (totalacc3) by the length of the test dataset (len(y_test)).

```
labels,pred,loss3,totalacc3=run_model(mymodel,test_dataloader,optimizer,False)
print("epoch passed ----------------------------------------------------------------------------------------------------")
print("total loss",loss3)
print("accuracy",totalacc3/len(y_test))
```

```
losssss tensor(6.4700, device='cuda:0', grad_fn=<AddBackward0>)
---------------------------------------------------------------
accuracy:  0.23828125
---------------------------------------------------------------
losssss tensor(6.2395, device='cuda:0', grad_fn=<AddBackward0>)
---------------------------------------------------------------
accuracy:  0.2109375
---------------------------------------------------------------
losssss tensor(6.3971, device='cuda:0', grad_fn=<AddBackward0>)
---------------------------------------------------------------
epoch passed ----------------------------------------------------------------------------------------------------
total loss 6.439489841461182
accuracy 0.249
```

Fourteenth. We display the train and validation accuracy per epoch.

```
print(train_accuracy)
print(val_accuracy)
```

```
[0.20270676691729322, 0.24714285714285714, 0.2469924812030075, 0.2505263157894737, 0.25, 0.2509774436090226, 0.24962406015037594, 0.24894736842105264, 0.24962406015037594, 0.24902255639097745]
[0.24035087719298245, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244, 0.25035087719298244]
```

Fifteenth. We define another custom dataset class named myDataset2 that is inherited from the Dataset class in PyTorch. This class is designed to work with a specific data format, which includes images, questions, answers, and their associated labels.

```
class myDataset2(Dataset):
  def __init__(self, array,labels1,labels2,tr=False):
    self.data = array
    self.labels1=labels1
    self.labels2=labels2
    self.tr=tr

  def __getitem__(self, index):
        item=self.data[index]
        answerable = item['answerable']
        image = item['image']
        question = item['question']
        answer_type = item['answer_type']
        answers = item['answers']
        answer1  = answers[0]['answer']
        answer2  = answers[1]['answer']
        answer3  = answers[2]['answer']
        answer4  = answers[3]['answer']
        answer5  = answers[4]['answer']
        answer6  = answers[5]['answer']
        answer7  = answers[6]['answer']
        answer8  = answers[7]['answer']
        answer9  = answers[8]['answer']
        answer10 = answers[9]['answer']

        yes=[]
        maybe=[]
```

- The __init__ method is the constructor for the class. It initializes the dataset by taking in an array of data, labels1, labels2, and a boolean flag tr indicating whether the dataset is used for training (True) or not (False). The data, labels, and the flag are stored as attributes of the class.
- The __getitem__ method is implemented to retrieve an item from the dataset given an index. It retrieves the item at the specified index from the data array. The item contains various attributes such as answerable, image, question, answer_type, and answers. It processes the answers to categorize them into "yes" and "maybe" lists based on their "answer_confidence" attribute. It then performs some image randomized augmentation based on the random value zz and preprocesses the image and question using the CLIP model. Finally, it returns the processed image and question features as datrain and the answerable label as datalabel1.

```
for answer in answers:
    if answer['answer_confidence']=="yes":
        yes.append(answer)
    else:
        maybe.append(answer)
use=[]
if len(yes)>0:
    use=yes
else:
    use=maybe
common=[]
i=0
for t in use:
    common.append(use[i]['answer'])
    i=i+1

with torch.no_grad():
    image_data = Image.open('/kaggle/input/vizwiz/vizwiz_data_ver1/data/Images/'+image)
    if(self.tr):
        zz=random.randint(0,6)
```

```
                if(zz==0):
                    image_data = image_data.transpose(Image.FLIP_LEFT_RIGHT)
                elif(zz==1):
                    image_data = image_data.transpose(Image.FLIP_TOP_BOTTOM)
                elif(zz==2):
                    image_data = image_data.transpose(Image.ROTATE_90)
                elif(zz==3):
                    enhancer = ImageEnhance.Brightness(image_data)
                    image_data = enhancer.enhance(1.5)
                elif(zz==4):
                    enhancer = ImageEnhance.Brightness(image_data)
                    image_data = enhancer.enhance(0.5)

            image_input = preprocess(image_data).unsqueeze(0).to(device)
            text_input = clip.tokenize([question]).to(device)

        with torch.no_grad():
            image_features = model.encode_image(image_input)
            text_in_features = model.encode_text(text_input)
            channel1=torch.tensor (image_features).float().clone().detach()
            channel2=torch.tensor (text_in_features).float().clone().detach()
        datrain = torch.cat((channel1.unsqueeze(1), channel2.unsqueeze(1)), dim=1).clone().detach().to(device)
        datalabel1=torch.tensor([answerable]).clone().detach().to(device)

        return datrain, datalabel1

def __len__(self):
    return len(self.data) # of how many examples(images?) you have
```

- The __len__ method returns the length of the dataset, which corresponds to the total number of examples or data points in the dataset.

Sixteenth. We define a custom neural network model class named Model2 that inherits from the nn.Module class in PyTorch. this model architecture applies linear transformations, normalization, dropout, and attention mechanism to process the input data and generate predictions for a specific label (label1).

- The __init__ method is the constructor for the class. It initializes the model by defining various layers and operations. It includes linear layers, dropout, activation functions (ReLU and Sigmoid), normalization layer (LayerNorm), attention layer (MultiheadAttention), and output layer (Linear).
- The forward method implements the forward pass of the model. It takes an input tensor x as the input to the model. The input tensor is reshaped to have the shape (1 dimension length of the total number of variables in the two concatenated encodings) to match the expected input shape. Then, the input tensor is passed through the linear layer (self.linear1), followed by

normalization (self.norm), dropout regularization (self.dropout), and activation function (self.activation). The tensor is then transposed and passed through the attention layer (self.attention) to capture the relationships between different elements in the sequence. The tensor is transposed back and passed through the output layer (self.output_layer1) to generate the predictions for label1.

```
class Model2(nn.Module):
    def __init__(self):
        super(Model2, self).__init__()
       # Linear layers
        self.linear1 = nn.Linear(768 * 2, 256)
#         self.linear2 = nn.Linear(4, out1)
        self.linear3 = nn.Linear(256,512)
        self.linear4 = nn.Linear(512,256)

        self.dropout = nn.Dropout (p=0.5)
        self.activation = nn.ReLU()
        self.activation2 = nn.Sigmoid()
        self.softmax =nn.Softmax ()
        # Normalization layer
        self.norm = nn.LayerNorm(256)

        # Attention layer
        self.attention = nn.MultiheadAttention(embed_dim=256, num_heads=4)

        # Output layer
        self.output_layer1 = nn.Linear(256,1)
```

```
def forward(self, x):
    x = torch.tensor(x).clone().detach().to(device)
    # Reshape input to (batch_size, sequence_length, embedding_size)
    x = x.view(x.size(0), -1, 768 * 2)
    # Apply linear layers
    x = self.linear1(x)


    # Apply normalization layer
    x = self.norm(x)
    x = self.dropout(x)
    x = self.activation(x)


    # Transpose to (sequence_length, batch_size, embedding_size)
    x = x.transpose(0, 1)
    # Apply attention layer
    x, _ = self.attention(x, x, x)
    # Transpose back to (batch_size, sequence_length, embedding_size)
    x = x.transpose(0, 1)


    # Apply output layer
    label1 = self.output_layer1(x)

    return label1
```

```
customDataset2=myDataset(data,labels1,labels2)
customDataset_train2=myDataset2(X_train,labels1,labels2,True)
customDataset_val2=myDataset2(X_val,labels1,labels2,False)
customDataset_test2=myDataset2(X_test,labels1,labels2,False)
whole_dataloader2 = DataLoader(customDataset2, batch_size=256,shuffle=True, num_workers=0)
train_dataloader2 = DataLoader(customDataset_train2, batch_size=256,shuffle=True, num_workers=0)
val_dataloader2 = DataLoader(customDataset_val2, batch_size=256,shuffle=True, num_workers=0)
test_dataloader2 = DataLoader(customDataset_test2, batch_size=256,shuffle=True, num_workers=0)
```

Seventeenth. defines and creates data loaders for a custom dataset using the DataLoader class in PyTorch. These data loaders will be used to efficiently load the data in batches during the training, validation, and testing phases of the model.

Eighteenth. We define a function named run_model2 that runs the training or evaluation loop for a given model on a specified data loader and returns loss and accuracy. Here's what the function does:

- model: The neural network model to be trained or evaluated.

- dataloader: The data loader containing the dataset to be used for training or evaluation.
- optimizer: The optimizer used to update the model's parameters during training.
- train: A boolean flag indicating whether the model should be trained (True) or evaluated (False).

The function performs the following steps:

- Sets the model to training mode (model.train()) if train=True, or evaluation mode (model.eval()) if train=False.
- Initializes variables for storing predictions, labels, and loss.
- Defines the loss function as L1 loss (nn.L1Loss()).
- Iterates over the batches in the data loader:
  a. Clears the gradients of the optimizer (optimizer2.zero_grad()).
  b. Passes the data through the model to get the output (output1).
  c. Calculates the loss between the predicted output and the ground truth label.
  d. Updates the total loss by adding the current batch loss.
  e. If train=True, performs backpropagation to compute gradients and updates the model's parameters (loss_.backward() and optimizer2.step()). Additionally, it adjusts the learning rate using the scheduler (scheduler2.step(loss_)).
  f. Appends the predicted output (output1) and the ground truth label (label1) to the corresponding lists.
  g. Calculates the accuracy by comparing the predicted output with the ground truth label and updates the acc and totalacc variables.
  h. Prints the accuracy and loss for the current batch.

Then return the labels, predictions, average loss per batch, and total accuracy.

```
def run_model2(model,dataloader, optimizer,train = True ):
    if train:
        model.train()
    else:
        model.eval()
    totalacc=0
    pred = []
    labels = []
    loss = nn.L1Loss()
    total_loss = 0
    for (data, label1) in dataloader:
        optimizer2.zero_grad()
        output1 = model(data)
        loss_ = loss(torch.squeeze(output1.float()), torch.squeeze(torch.tensor(label1).to(device).float()))

        total_loss += loss_.item()
        if (train):
            loss_.backward()
            scheduler2.step(loss_)

        optimizer2.step()
        pred += [output1]
        labels += [label1]
        acc=0
        i=0
```

```
    for l in label1:
        if (label1[i][0].float().item()==1 and output1[i][0].float().item()>0.5) or (label1[i][0].float().item()==0 and output1[i][0].float().item()<0.5):
            acc+=1
            totalacc+=1
        i+=1
    print("accuracy: ",acc/256)
    print("------------------------------------------------------------------")
    print("losssss",loss_)
    print("------------------------------------------------------------------")
  return labels,pred, total_loss/len(dataloader),totalacc
```

Nineteenth. We trained a neural network model (mymodel2) for the specified number of epochs and collects the training and validation metrics for each epoch. using the training and validation data loaders. Which can be used to analyze the performance of the model during training and evaluate its effectiveness.

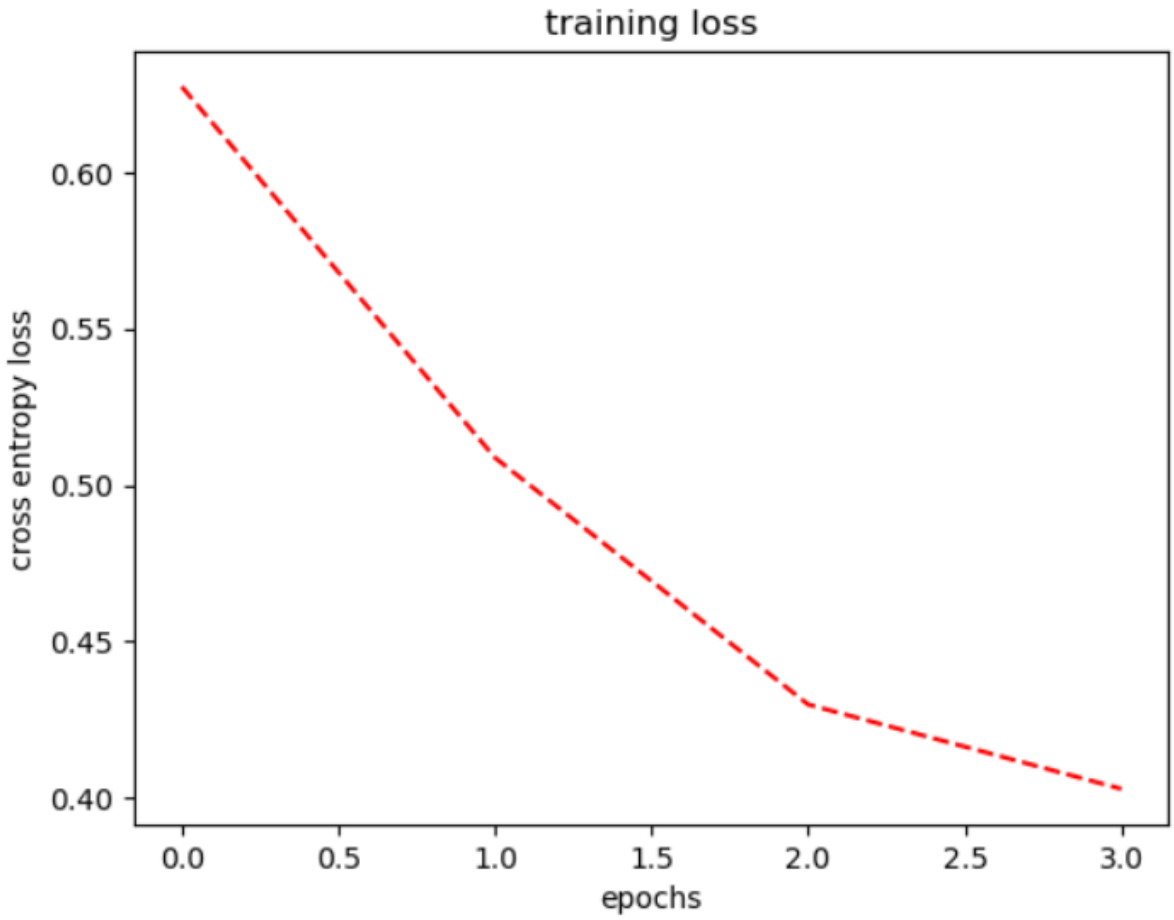
- Create an instance of the Model2 class (mymodel2) and move it to the specified device (device).
- Define variables to store training and validation metrics (train_accuracy2, val_accuracy2, train_loss2, val_loss2).

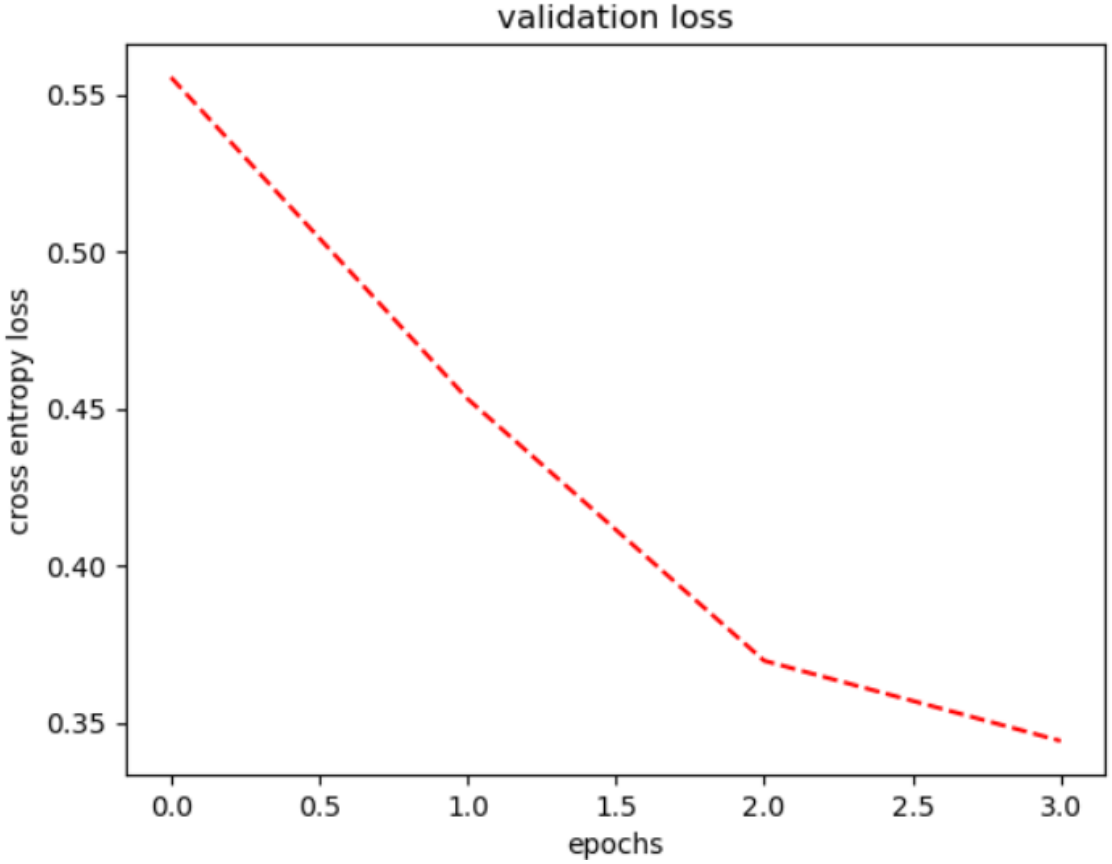
- Define an optimizer (optimizer2) using the Adam optimizer with a specified learning rate (lr) and weight decay (weight_decay).
- Define a scheduler (scheduler2) that reduces the learning rate when the validation loss plateaus using the ReduceLROnPlateau scheduler.
- Start a loop over the specified number of epochs (epoch).
- Call the run_model2 function to train the model (mymodel2) on the training data (train_dataloader2). It returns the ground truth labels (labels2), predicted outputs (pred2), loss per batch (loss21), and total accuracy (totalacc21).
- Call the run_model2 function to evaluate the model (mymodel2) on the validation data (val_dataloader2) by setting train=False. It returns the same metrics for the validation set (labels2, pred2, loss22, totalacc22).
- Append the training loss, training accuracy, validation loss, and validation accuracy to the corresponding lists (train_loss2, train_accuracy2, val_loss2, val_accuracy2).

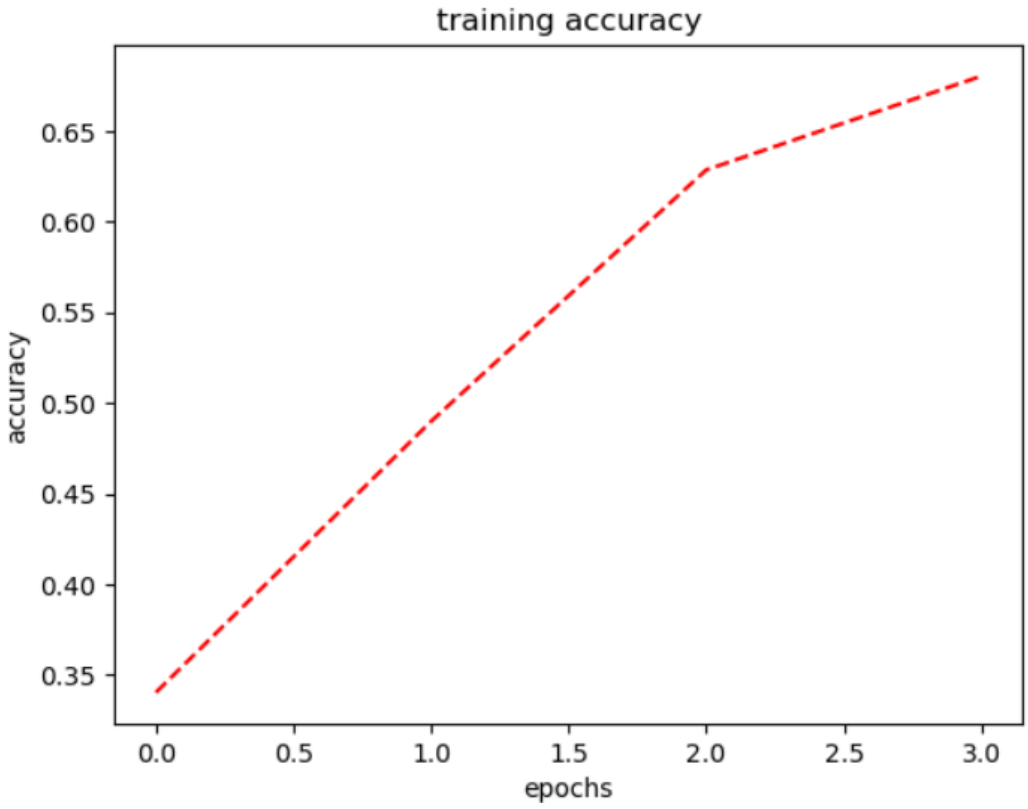
```
mymodel2=Model2().to(device)
epoch = 4
train_accuracy2=[]
val_accuracy2=[]
train_loss2=[]
val_loss2=[]
optimizer2 = torch.optim.Adam(mymodel2.parameters(), lr=0.00001, weight_decay=.1)
scheduler2 = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer2, patience=5, factor=.7, threshold=1e-4)

for e in range(epoch):
    labels2,pred2,loss21,totalacc21=run_model2(mymodel2,train_dataloader2,optimizer2)
    print("train passed ----------------------------------------------------------------------------------------------------")
    labels2,pred2,loss22,totalacc22=run_model2(mymodel2,val_dataloader2,optimizer2,False)

  # calculate acc, f1 score, recall ......
    print("epoch passed ----------------------------------------------------------------------------------------------------")
    print("total loss",loss21)
    train_loss2.append(loss21)
    print("train accuracy",totalacc21/len(y_train))
    train_accuracy2.append(totalacc21/len(y_train))
    print("total val loss",loss22)
    val_loss2.append(loss22)
    print("accuracy val",totalacc22/len(y_val))
    val_accuracy2.append(totalacc22/len(y_val))
```

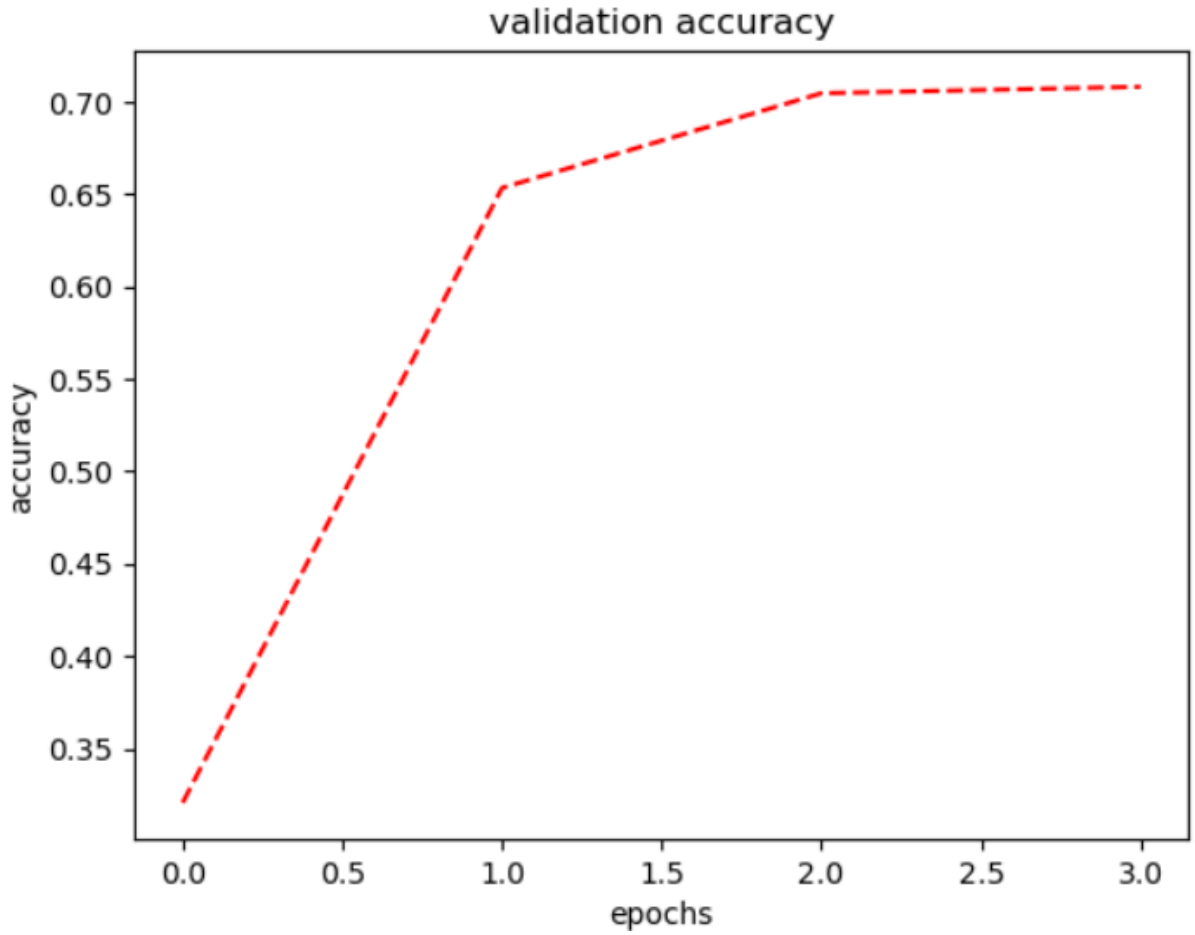
Twentieth. We visually represent the training loss (epochs vs cross entropy loss), validation loss (epochs vs cross entropy loss), training accuracy (epochs vs accuracy) and validation accuracy (epochs vs accuracy).

```python
import matplotlib.pyplot as plt

def plot_broken_line(data,y,t):
    x = range(len(data))  # Generate x-axis values as indices of the list
    plt.plot(x, data, 'r--')  # Plot the data as a broken line with red dashed line style
    plt.xlabel('epochs')
    plt.ylabel(y)
    plt.title(t)
    plt.show()

plot_broken_line(train_loss2,"cross entropy loss","training loss")
plot_broken_line(val_loss2,"cross entropy loss","validation loss")
plot_broken_line(train_accuracy2,"accuracy","training accuracy")
plot_broken_line(val_accuracy2,"accuracy","validation accuracy")
```




validation loss
cross entropy loss
0.55
0.50
0.45
0.40
0.35
0.0
0.5
1.0
1.5
2.0
2.5
3.0
epochs


training accuracy
accuracy
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.0
0.5
1.0
1.5
2.0
2.5
3.0
epochs



Twenty-first. We evaluated the trained model mymodel2 on the test dataset using the test_dataloader2 dataloader.

- Call the run_model2 function to evaluate the model on the test dataset. The mymodel2 is passed as the model to be evaluated, test_dataloader2 is the dataloader containing the test data, and False indicates that the model should be in evaluation mode. The function returns the labels, predictions, loss, and total accuracy for the test dataset, which are stored in the variables labels2, pred2, loss32, and totalacc32, respectively.
- print the total loss calculated for the test dataset.
- Calculate and print the accuracy of the model on the test dataset. It divides the totalacc32 (total number of correct predictions) by the length of the y_test (number of samples in the test dataset) to get the accuracy.

```
labels2,pred2,loss32,totalacc32=run_model2(mymodel2,test_dataloader2,optimizer2,False)
print("epoch passed ----------")
print("total loss",loss32)
print("accuracy",totalacc32/len(y_test))
```

```
/tmp/ipykernel_28/3218481495.py:70: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.clone().detach() or sou
().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  channel1=torch.tensor (image_features).float().clone().detach()
/tmp/ipykernel_28/3218481495.py:71: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.clone().detach() or sou
().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  channel2=torch.tensor (text_in_features).float().clone().detach()
accuracy:  0.69140625
------------------------------------------------------------
lossss tensor(0.3442, device='cuda:0', grad_fn=<MeanBackward0>)
------------------------------------------------------------
epoch passed ----------
total loss 0.34419387578964233
accuracy 0.708
```

Twenty-second. We display the train and validation accuracy.

```
print(train_accuracy2)
print(val_accuracy2)
```

```
[0.3404511278195489, 0.4899248120300752, 0.6285714285714286, 0.6803007518796993]
[0.32070175438596493, 0.6533333333333333, 0.7045614035087719, 0.7080701754385965]
```

➢ Bonus:

I. Latex Report:

We wrote another report in latex, and the following is the code that we generated the report using.

```
\documentclass{article}

\usepackage[a4paper, left=1.5cm, right=1.5cm, top=1.5cm, bottom=1.5cm]{geometry}

\usepackage{lipsum} % for placeholder text (remove this line)

% Document Information

\title{\textbf{Assignment Four \\ Visual Question Answering}}

\author{\textbf{Omar Walied 7058 , Hend Khaled 6986 , Habiba Hefny 6939}}

\date{Due: 25 June 2023}

\begin{document}

\maketitle

\section{Introduction}

VQA is a rapidly advancing field that merges computer vision and natural language processing to enable machines to understand and
```

answer questions about images. Its applications range from robotics to autonomous vehicles, virtual assistants, and augmented reality, promising a future where machines can comprehend and interact with visual information in a human-like manner.

\section{Problem Description}

Our focus is on VizWiz dataset, which is designed for training models to assist visually impaired individuals. The dataset includes images and spoken questions, providing a real-world context for visually impaired people, aiming to improve their understanding of visual information by using VQA techniques. VizWiz-VQA contains diverse answers crowd-sourced from the community. We analyze the dataset's characteristics and explore VQA methodologies to enhance visual perception for visually impaired individuals. By training models on this dataset, we aim to improve accessibility and inclusivity, ultimately enhancing the quality of life for visually impaired individuals.

\section{Code Explaination}

\subsection{Notebook Setup and Dependencies}

\begin{itemize}

  \item Creating a new Kaggle notebook

  \item Adding the dataset and installing necessary dependencies and libraries

  \item Importing libraries such as PyTorch, torchvision, numpy, cv2, matplotlib, etc.

\end{itemize}

\subsection{Device Configuration}

\begin{itemize}

  \item Setting the device variable based on the availability of a CUDA-enabled GPU

  \item Choosing between "cuda" and "cpu" for computations

\end{itemize}

\subsection{Reading and Processing JSON Data}

\begin{itemize}

```
  \item Reading the contents of a JSON file and storing it as a string

  \item Converting the string into an object based on the JSON
structure

  \item Processing the data and generating two sets of labels (labels1
and labels2)

  \item Plotting the frequency of each letter in the label string

\end{itemize}

\subsection{Custom Dataset Class}

\begin{itemize}

  \item Defining a custom dataset class named "myDataset"

  \item Inheriting from the torch.utils.data.Dataset class

  \item Implementing methods: init, getitem, and len

  \item Initializing the dataset and storing data, labels, and a train
flag

  \item Retrieving an item from the dataset based on the index

  \item Returning the length of the dataset

\end{itemize}

\subsection{Neural Network Model}

\begin{itemize}

  \item Building a neural network model inherited from torch.nn.Module
class

  \item Defining layers such as linear, activation, normalization,
dropout, and attention

  \item The model architecture processes input data and generates
predictions for two labels

  \item Initializing the model in the init method and implementing the
forward pass in the forward method

\end{itemize}

\subsection{Data Splitting and Dataloaders}

\begin{itemize}
```

```
  \item Splitting the data into training, validation, and testing sets

  \item Creating dataloaders to efficiently load the data during
training, validation, and testing

\end{itemize}

\subsection{Model Training and Evaluation}

\begin{itemize}

  \item Defining the run model function for the training and
evaluation loop

  \item Running the model on a specified dataloader and updating the
parameters with an optimizer

  \item Recording training and validation loss and accuracy values for
each epoch

  \item  Visualizing the training and validation loss, and training
and validation accuracy

\end{itemize}

\subsection{Model Evaluation on Test Data}

\begin{itemize}

  \item Evaluating the performance of the trained model on the test
dataset

  \item Calculating loss and accuracy metrics

\end{itemize}

\subsection{Continue of Evaluation (Answerability)}

\begin{itemize}

  \item Define a new custom dataset class. It is designed for
evaluating the model's answerability.

  \item we perform the previous steps (3.4, 3.5, 3.6, 3.7, 3.8)

\end{itemize}

\section{Conclusion}
```

This report focuses on addressing the challenges faced by visually impaired individuals through visual question answering (VQA) techniques applied to the VizWiz dataset. Custom dataset and model classes were developed to handle the dataset and perform VQA tasks, incorporating

various techniques such as linear transformations, normalization, dropout, and attention mechanisms. Data loaders were created to efficiently handle data during training, validation, and testing. The training process involved running the model on the data loader, collecting metrics, and visualizing the progress. The trained model was evaluated on a test dataset, aiming to improve the accessibility and comprehension of visual information for visually impaired individuals. The approach aimed to combine computer vision and natural language processing to enhance their quality of life.

\end{document}

II. Model Evaluation on Unseen Image-Question Pairs:

The directory path containing the images is specified through the 'image_directory' variable. We first loop on the images in the folder and copy the directory of the first 50 images, and the code defines a default question: "What color is the object in the image?", which will be used for each image, such that the image path and the question are added to the unseen_data list as a dictionary item.

```
# Specify the directory path containing the images
image_directory = '/kaggle/input/visual-question-answering-computer-vision-nlp/dataset/images'

# Initialize the unseen_data list
unseen_data = []
i=50
# Iterate over the images in the directory
for filename in os.listdir(image_directory):
    i=i-1
    if i<0:
        break
    if filename.endswith('.png'):  # Filter only image files
        image_path = os.path.join(image_directory, filename)
        question = 'What color is the object in the image?'
        data_item = {
            'image_path': image_path,
            'question': question
        }
        unseen_data.append(data_item)
```

The loop then iterates over each item in the unseen_data list. The image path and question are extracted from the current item. The image is opened using Image.open(image_path) and assigned to the image_data variable. Next, the image is preprocessed then converted to a tensor and moved to the appropriate device (specified by device). Considering the question, it is tokenized using the

CLIP model's tokenize() function, which converts the question into a tensor representation compatible with the model.

The image and text features are encoded using the CLIP model. The image features are obtained by calling model.encode_image(image_input), where model is the CLIP model instance. Similarly, the text features are obtained by calling model.encode_text(text_input).

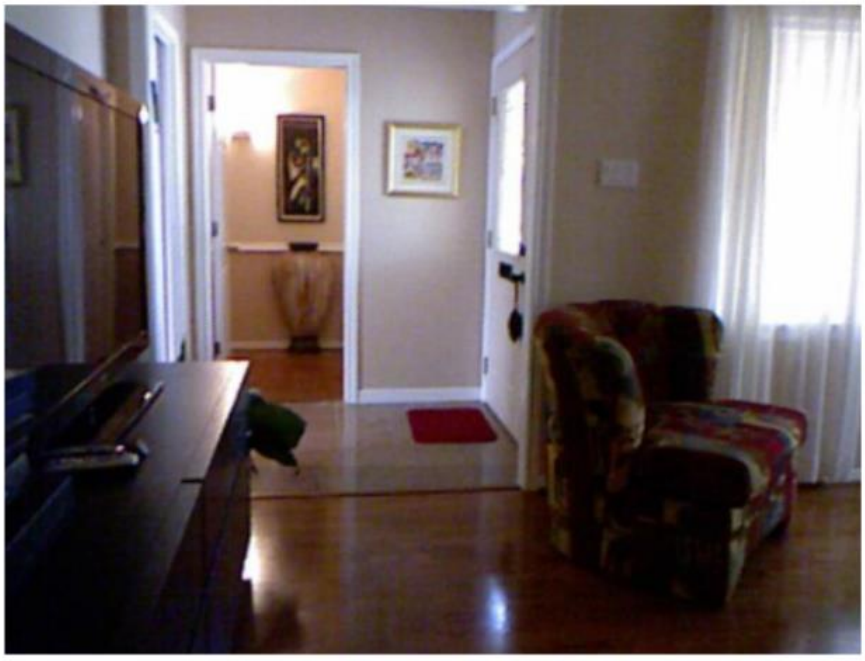
The image and text features are concatenated using torch.cat() to create a single tensor representation. This tensor is then assigned to the x variable.

The x tensor is passed through the mymodel (presumably your custom model) to perform the inference. The outputs are assigned to output1 and output2. The predicted answer type and category are determined by finding the maximum values in output1 and output2, respectively, and mapping them to their corresponding labels (labels1 and labels2).

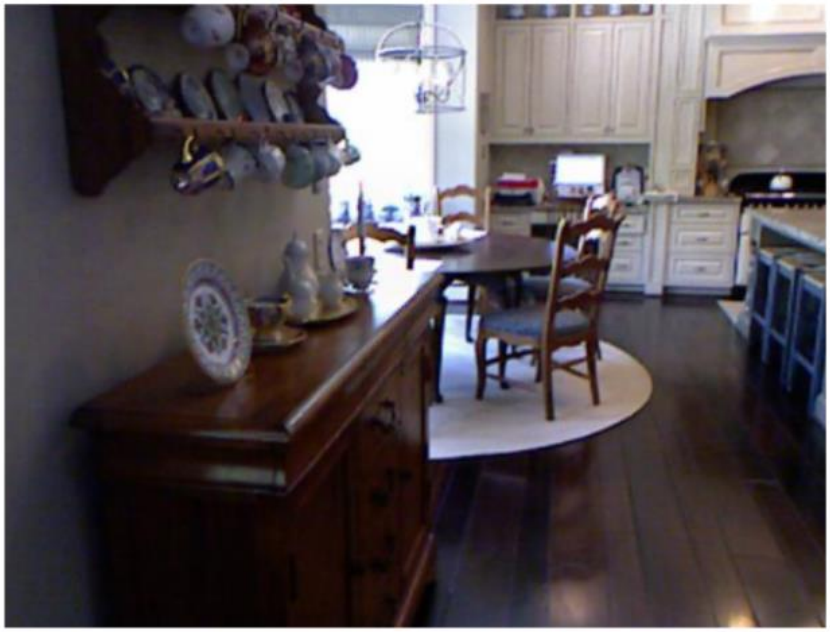
```
#unseen_data = [...]  # List of unseen image-question pairs

with torch.no_grad():
    for item in unseen_data:
        image_path = item['image_path']
        question = item['question']

        image_data = Image.open(image_path)
        image_input = preprocess(image_data).unsqueeze(0).to(device)
        text_input = clip.tokenize([question]).to(device)

        image_features = model.encode_image(image_input)
        text_features = model.encode_text(text_input)
        concatenated_features = torch.cat((image_features.unsqueeze(1), text_features.unsqueeze(1)), dim=1)
        x = torch.tensor(concatenated_features, dtype=torch.float32).clone().detach().to(device)

        output1, output2 = mymodel(x)

        predicted_answer_type = labels1[torch.argmax(output1).item()]
        predicted_answer_category = labels2[torch.argmax(output2).item()]

         # Print image
        plt.imshow(image_data)
        plt.axis('off')
        plt.show()
```

```
print("Image:", image_path)
print("Question:", question)
print("Predicted Answer Type:", predicted_answer_type)
print("Predicted Answer Category:", predicted_answer_category)
print("--------------------")
```

```
Image: /kaggle/input/visual-question-answering-computer-vision-nlp/dataset/images/image1317.png
Question: What color is the object in the image?
Predicted Answer Type: other
Predicted Answer Category: unanswerable
--------------------
```



```
Image: /kaggle/input/visual-question-answering-computer-vision-nlp/dataset/images/image1268.png
Question: What color is the object in the image?
Predicted Answer Type: other
Predicted Answer Category: unanswerable
```



```
Image: /kaggle/input/visual-question-answering-computer-vision-nlp/dataset/images/image745.png
Question: What color is the object in the image?
Predicted Answer Type: other
Predicted Answer Category: unanswerable
--------------------
```

```
Image: /kaggle/input/visual-question-answering-computer-vision-nlp/dataset/images/image105.png
Question: What color is the object in the image?
Predicted Answer Type: other
Predicted Answer Category: unanswerable
```